

KURTOSIS

- AWDHOOT ASHUTOSH KUMAR PRASAD
B.TECH IN INFORMATION TECHNOLOGY
VJTI 2025-29

INTRODUCTION :

Hello, I'm Awdhoot Prasad and I have just completed my first year as a Information technology student at Veermata jijabai technological institute. Over the course of one year I have built a strong foundation in C++, java and python. I really enjoy solving problems and learning new things. I have also started learning Data Structures and Algorithms. I have chosen this project because I really enjoy learning physics and on top of that for this project we are going to combine both physics and Artificial Learning for the analysis of the health of machines.

ABSTRACT :

Machines have been making human life easier from the day they have arrived or built. As they are made of metal components over the course of time they will definitely corrode or may have some part broken. Unexpected failures can lead to costly downtime, safety risks, and massive operational losses. Factories commonly lose ₹2–10 lakhs per hour.

Modern industries rely on thousands of motors, bearings, turbines, and rotating systems that power everything from manufacturing plants to transportation networks. But these machines give us warnings in the form of vibrations. So analysis of these vibrations can reveal the health of machines.

Our motive is to build an end to end pipeline for Predictive Failure Analysis of machines by studying the vibrations signal.

Contents :

1. project objective
2. Theory and Approach
 - A. Phase 1 -
 1. Time series analysis
 2. Frequency domain analysis
 - B. Phase 2 (Deep learning) -
 1. 1D signal to 2D image
 2. TCN
 3. Normalizing flows
3. Roadmap and workflow
4. References

Project Objective :

- Build a pipeline using CWRB bearing dataset to process high frequency 1D vibrations signals.
- Develop a classical machine learning baseline utilizing fast fourier transforms to isolate characteristic fault frequencies.
- Prove model robustness through cross-dataset transfer testing, ensuring the model identifies true mechanical faults rather than memorizing laboratory background noise.
- Explore spatial representation learning by transforming time series signals into 2D visuals formats for convolutional Neural network(CNN) anomaly detection.
- Exploring how TCN and how Normalizing Flows can help us analyze the 1D signal to detect anomalies.

Theory and Approach :

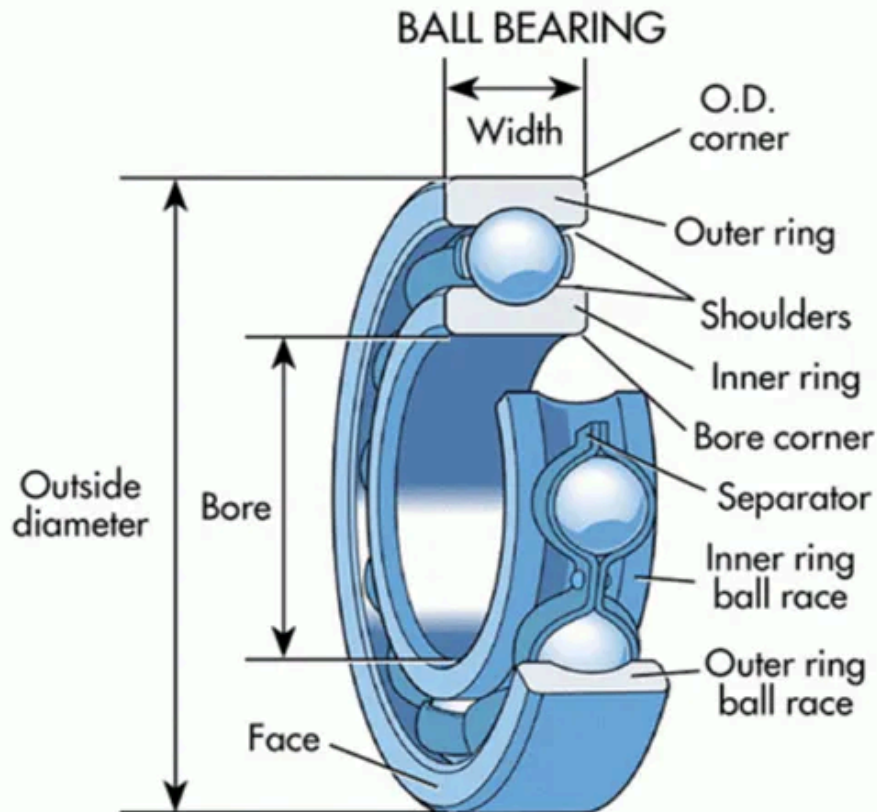
Phase 1: Physics and feature engineering way -

A. Bearings -

A **bearing** is a machine element that constrains relative motion to only the desired motion and reduces friction between moving parts. The design of the bearing may provide free movement of the moving part or free rotation around the axis. The most important work of the bearing is to minimize friction.



B. Parts of rolling bearing elements -



A rolling element bearing is consisting of the following components,

1. Inner race/ring
2. Outer race/ring
3. Cage/separator
4. Rolling elements (balls/rollers)

C. Bearing faults frequency calculations -

- BD — Ball diameter.

- NB — Number of balls (rolling elements).
- θ — Contact angle
- PD — Pitch diameter ($PD = (D1 + D2) / 2$)

Fundamental train frequency (FTF)

$$FTF (Hz) = \frac{N}{60} \cdot \frac{1}{2} \cdot \left(1 - \frac{B_D}{P_D} \cdot \cos \theta \right)$$

Ball pass frequency outer (BPFO)

$$BPFO (Hz) = \frac{N}{60} \cdot \frac{N_B}{2} \cdot \left(1 - \frac{B_D}{P_D} \cdot \cos \theta \right)$$

Ball pass frequency inner (BPFI)

$$BPFI(Hz) = \frac{N}{60} \cdot \frac{N_B}{2} \cdot \left(1 + \frac{B_D}{P_D} \cdot \cos \theta \right)$$

Ball spin frequency (BSF)

$$BSF (Hz) = \frac{P_D}{2 \cdot B_D} \cdot \left(\frac{N}{60} \left(1 - \frac{B_D^2 \cos^2 \theta}{P_D^2} \right) \right)$$

D. Vibrational analysis of bearings -

1. Time-Domain analysis -

The time domain depicts dynamic responses that are influenced by many system components' interactions. It aids in the detection of any rotating machinery system issue by analyzing the amplitude and phase information of the vibration time signal.

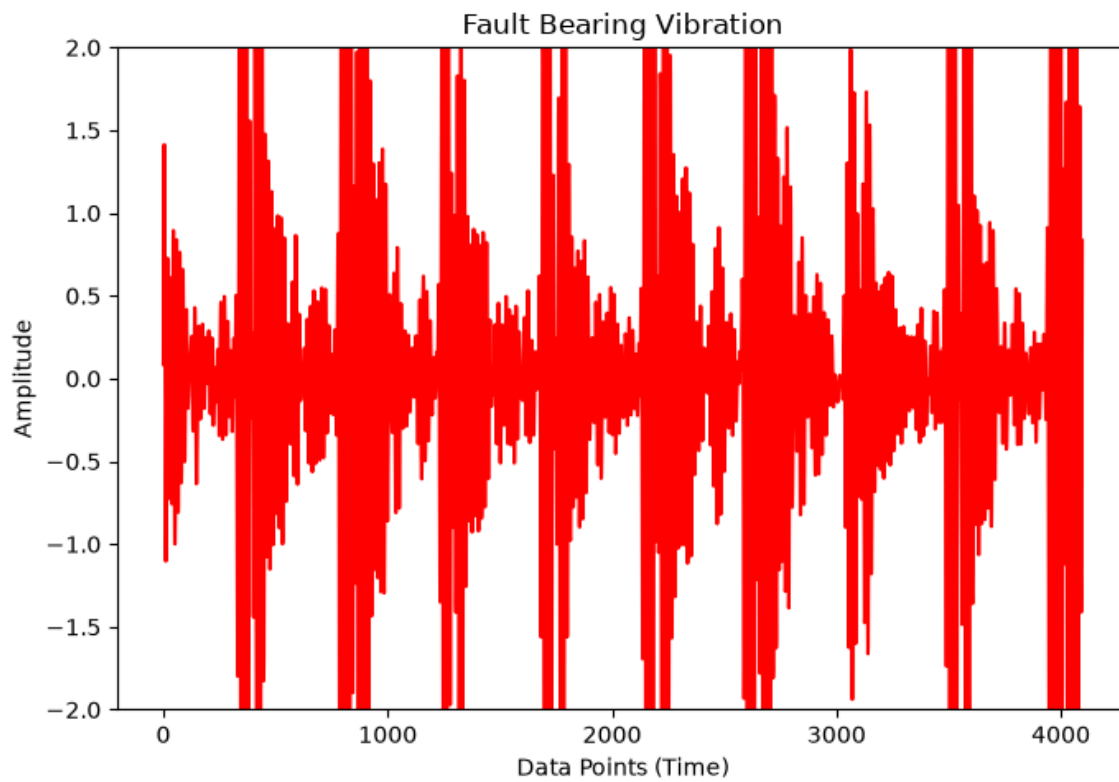
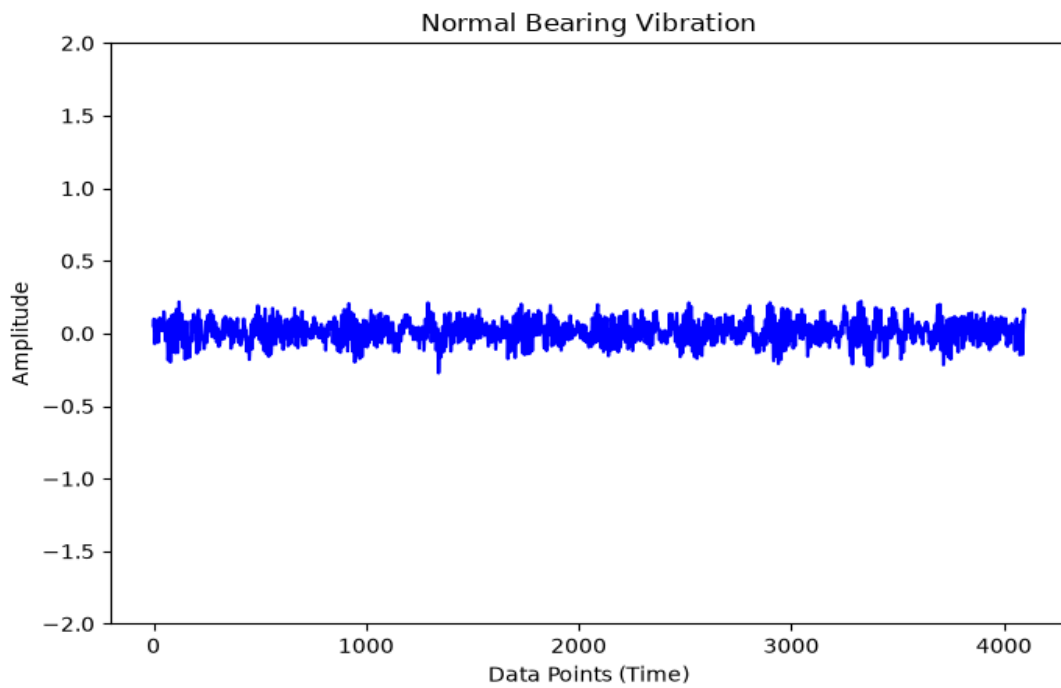
The time domain of the signal holds the very important parameters of the signal that can be used to analyze the signal

Data Collections -

Converting CWRU dataset and collecting vibration signals, We will divide the signal into a discrete number of points (in powers of 2) that will create a one window and for that window we can calculate the parameters that are hiding inside the signals.

Feature extraction -

For each discrete window, statistical metrics (Parameters) are extracted to measure overall mechanical degradation. This includes the Root Mean Square (RMS) to quantify the overall energy of the vibration, and Kurtosis to isolate the impulsive, high-amplitude spikes characteristic of physical metal-on-metal impacts.



Problems with time domain analysis -

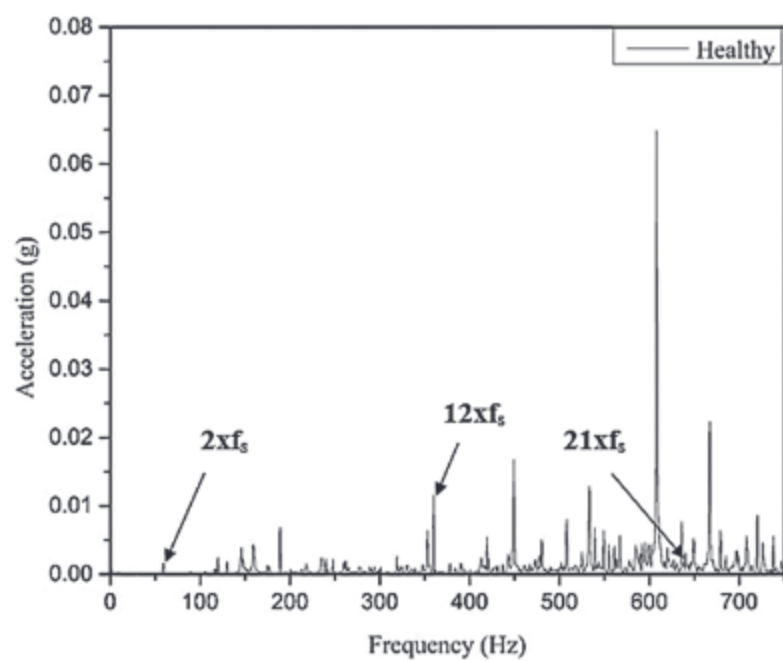
If we can analyze the signal using time series then why do we need to analyze frequency, Because it cannot be guaranteed that the environment in which the vibrations were recorded have a silent background the system may capture some unnecessary vibrations(noise) from the background and on the graph it will appear as spike and we may find think it is due to the bearings. But it was originally not from the bearing but just a noise from the background. Hence we need to analyze the frequency also.

How does Frequencies help in analyzing ?

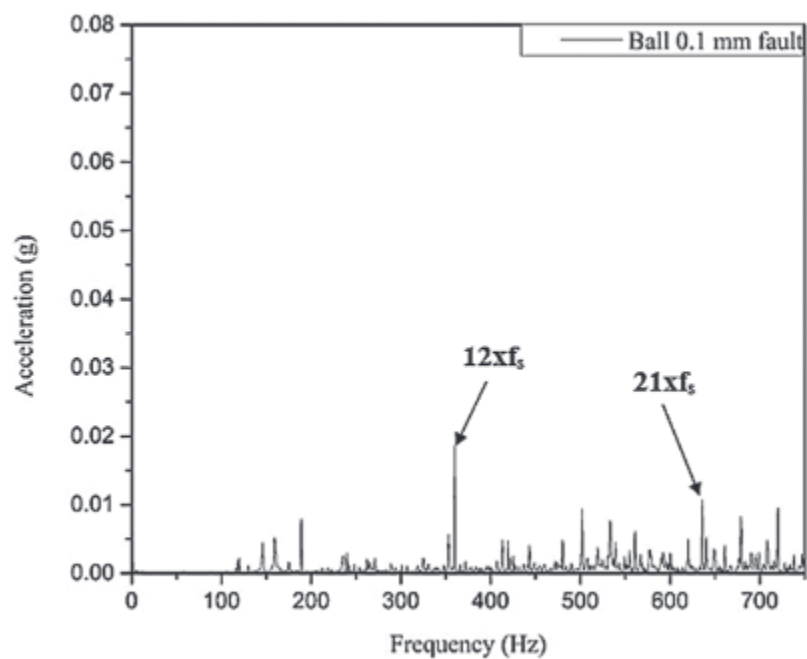
The bearings have a fault frequency that can be mathematically calculated from the formulas above mentioned. Frequencies like Ball pass frequency outer (BPFO), Ball pass frequency inner (BPFI) are fixed for bearings, so they can be mapped.

2. Frequency-Domain analysis -

Mechanical faults generate highly specific forced oscillations, we will utilize the Fast Fourier Transform to isolate characteristic fault frequencies. By mapping the mathematical formulas for Ball Pass Frequency Outer (BPFO) and Ball Pass Frequency Inner (BPFI), the algorithm dynamically searches the generated frequency spectrum and extracts the exact amplitudes at these physical resonance points.



(a) Healthy bearing



(b) Ball with 0.1 mm fault

Dividing the dataset into a fixed number of points (power of 2) to create a window and analyzing that window.

Creating dataframe from that collection (using pandas)-

sr.no	RMS	Kurtosis	BPFO	BPFI	label
-------	-----	----------	------	------	-------

Kurtosis can be calculated manually as well as using scipy library,
- from scipy.stats import kurtosis

Now after data collections and extracting useful features, we have to train the model, There are various model that can be used :

1. Supervised learning -

In supervised learning, we will label normal data as 0 and faulty data as 1 for classification.

a. KNN -

KNN classifies a new signal by comparing it to the 'K' most mathematically similar signals in the training dataset. It plots the extracted features in multi-dimensional space, calculates the distance (Euclidean distance) to the nearest known data points, and assigns the most common fault label among its neighbors.

b. Decisive Tree -

Decisive tree algorithm builds a flowchart like structure to classify faults based on simple threshold questions. Whenever a new

signal is processed it asks a question and then the path splits toward a classification. It is highly interpretable, making it easy to map.

c. Random Forest Classifier -

One of the best methods for classification of huge data, it builds hundreds of individual Decision Trees during training. Each tree operates on a random subset of the extracted features. When diagnosing a new signal, all the trees "vote" on the window health, it helps in reducing the variance and preventing the model from overfitting. It also does not need data to be scaled.

d. Support vector machine -

SVM maps the extracted features into a high dimensional mathematical space. It calculates the optimal geometric boundary (a hyperplane), the dimensionality of the plane is only changed by the edge points of the plane; anything inside the plane cannot change the boundary. It will separate the signal data and will create a plane.

e. XGBoost -

An advanced tree based ensemble that builds decision trees sequentially. Instead of building independent trees like a Random Forest, XGBoost specifically focuses on the residual errors of the previous trees. It continuously corrects its own mistakes, establishing a highly accurate boundary for complex, multi-class faults.

f. LogReg -

A foundational statistical model that maps the relationship between the extracted physical features (like RMS and kurtosis) and the probability of a binary outcome (Healthy and Faulty). It utilizes a sigmoid mathematical function to compress the output into a strict probability score between 0 and 1, making it highly efficient for baseline binary fault detection.

g. AdaBoost -

Similar to XGBoost, AdaBoost is a sequential ensemble technique. During training, if it incorrectly classifies a highly complex fault signal, it dynamically increases the "weight" of that specific signal. The next classifier built in the sequence is forced to pay more mathematical attention to that difficult fault, adapting over time to handle complex vibration anomalies.

h. GBoost -

It is the foundational algorithm behind XGBoost. It sequentially builds predictive models to optimize an arbitrary differentiable loss function. It minimizes the prediction errors of the bearing health state by moving in the direction of the mathematical gradient, making it an incredibly powerful baseline for tabular sensor data.

2. Unsupervised learning -

In unsupervised learning, we have to train models either on normal data or faulty data, since there is an abundance of normal data as compared to faulty data we train it mostly on normal data.

a. Isolation forest -

Because structural faults (massive amplitude spike from a metal impact) are mathematically rare and extreme, this algorithm isolates them using random data partitions. Abnormal vibration windows are isolated in very few tree splits, allowing the model to flag mechanical degradation purely based on a short mathematical "path length" without ever being trained on actual fault labels.

b. PCA (Principle component analysis) -

For many unsupervised learning, PCA is a critical unsupervised preprocessing step. When dozens of mathematical features are extracted from the vibration signal, PCA mathematically compresses this high-dimensional data into a few uncorrelated **"Principal Components"** that capture the maximum variance of the motor's physics. This drastically reduces the computational load and strips away background laboratory noise before the data is passed to a clustering algorithm or classifier.

c. K means -

It is one of the highest used unsupervised learning algorithms. After the vibration features are extracted, they are first compressed by PCA then K-Means attempts to partition the signals into distinct groups (K) based on their Euclidean distance in the feature space. For this project, K should be set according to the number of mechanical analyses parameters.

3. Deep learning for 1D signal-

a. 1D CNNs -

Bypassing manual Fourier transforms, this neural network slides a one-dimensional mathematical filter directly across the raw, windowed time-series array. It autonomously learns to extract localized spatial features, identifying the exact microsecond impact transients caused by rolling elements hitting a structural defect.

b. LSTMs -

RNNs are natively designed for time-series data. Long Short-Term Memory (LSTMs) is an improved version of the Recurrent Neural Network (RNNs) designed to capture long-term dependencies in sequential data. It uses a memory cell to store information over time, solving the limitations of traditional RNNs.

Phase 2: Deep learning way -

A. Converting 1D signals to 2D images -

Traditional time-series classification struggles to capture complex, non-linear mechanical degradation. To solve this problem, the 1D high-frequency vibrational signals will be transformed into 2D matrices using Gramian Angular Fields (GAFs). The 1D arrays will be mapped into polar coordinates systems. Generating a 2D image matrix that visually preserved the temporal correlations of the signal without the spectral leakage.

1. Loading the signals from the .mat file -

From the [Scipy.io](https://scipy.io) library we can directly load the .mat file.
`normal_data = scipy.io.loadmat(filename.mat).`

After loading the data we can search for the '___DE___' data because it is the main set that we have to work on. Upon finding it we load the signals from the data using a flatten function.

```
'X097_DE_time', 'X097_FE_time', 'X097RPM'])  
'X136_DE_time', 'X136_FE_time', 'X136RPM'])
```

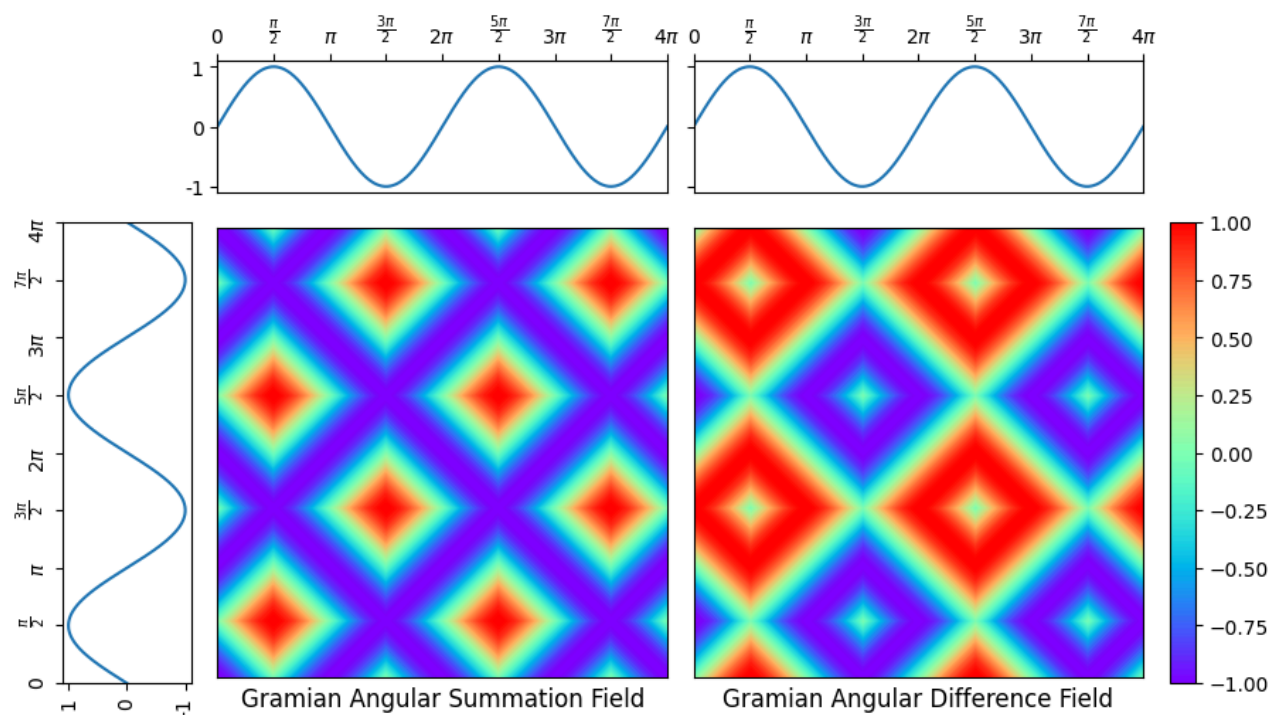
```
normal_signal = normal_data['___DE___ data'].flatten()
```

2. Converting 1D signals to 2D images -

Gramian angular field -

A Gramian angular field is an image obtained from a time series, representing some kind of temporal correlation

between each pair of values from the time series. Two methods are available: Gramian angular summation field and Gramian angular difference field. It is implemented as **pyts.image.GramianAngularField**.



Using Pytorch Deep Learning Framework -

Since we want to use CNN models for the image classification, we do not need to build the network from scratch, we can use built in frameworks such as pytorch.

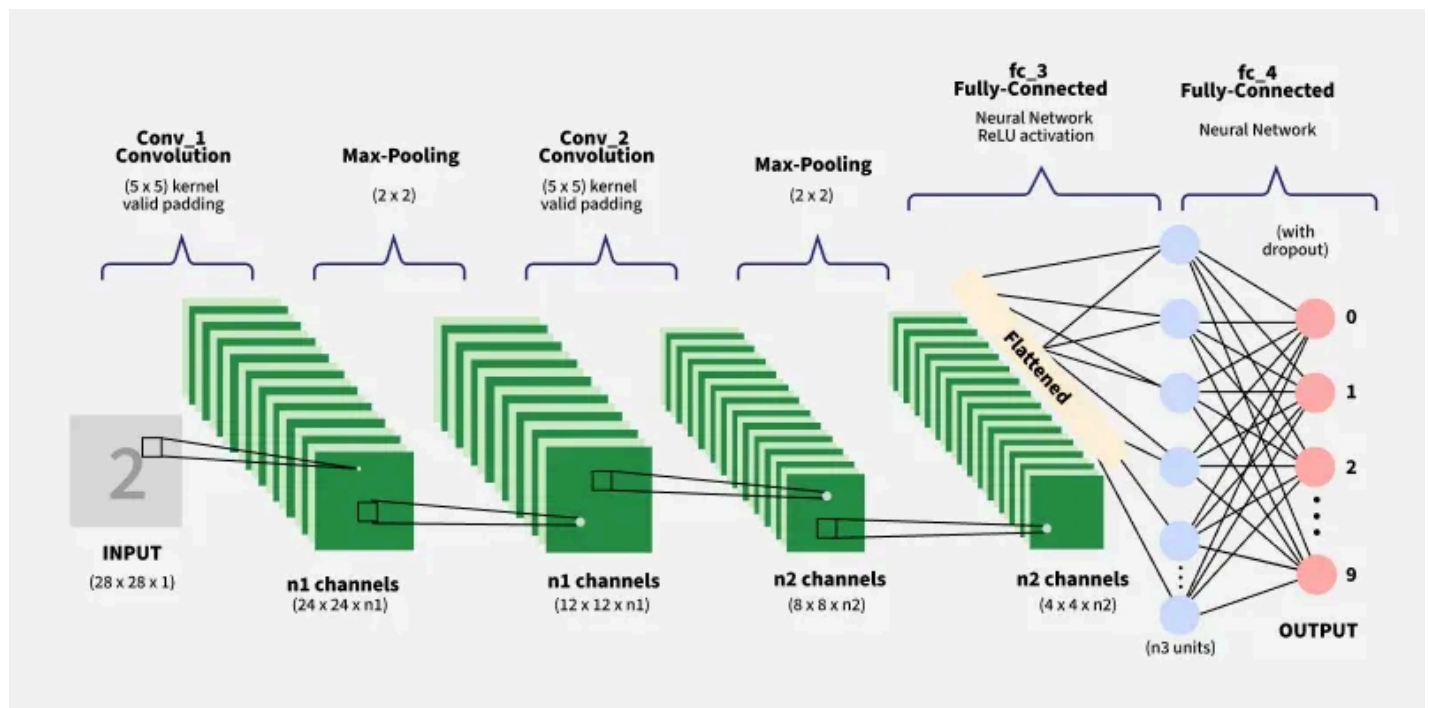
To prevent the severe memory restrictions associated with loading thousands of image matrices, we will utilize PyTorch Dataset and DataLoader classes. These classes act as algorithmic pointers, they mathematically collect the massive

100,000+ point continuous arrays and transform them into 128 GAF tensors when the GPU requests the batch.



Convolutional neural network (CNN) -

Convolutional Neural Networks (CNNs), are neural network architectures designed to process data with grid-like technology like images. They are used to extract features from the visual data.



Layers in CNNs -

1. Input layer : The input layer receives raw visual data and we convert it to standard sizes for classification and passes it to the network for processing.
2. Convolutional layer : The convolutional layer is responsible for the extraction of important features from input. It applies a kernels (filter) that slides over the image and computes the dot product between kernels and image, producing a feature map.
3. Activation layer : The activation layer inserts non linearity into the network by applying activation function to output of the convolutional layer.
ex - ReLu, Tahn, etc.
4. Pooling layer : The polling layer is used to reduce dimensionality of the feature map improving computational power, reduce memory space and prevent overfitting.
ex - max pooling, average pooling, etc.
5. Flattening - It converts the multi-dimensional feature maps into a one-dimensional vector. This vector is then passed to the fully connected layer for classification or regression.
6. Fully Connected Layer - The fully connected (dense) layer performs high-level reasoning using extracted features and produces the final classification scores.
7. Output Layer : The output layer converts final scores into probabilities using activation functions like Sigmoid (binary classification) or Softmax (multi-class classification).

Now we can build the pipeline by building CNN networks from scratch or we can utilize pre-built models like ResNet-18, VGG-16, efficient-net, etc.

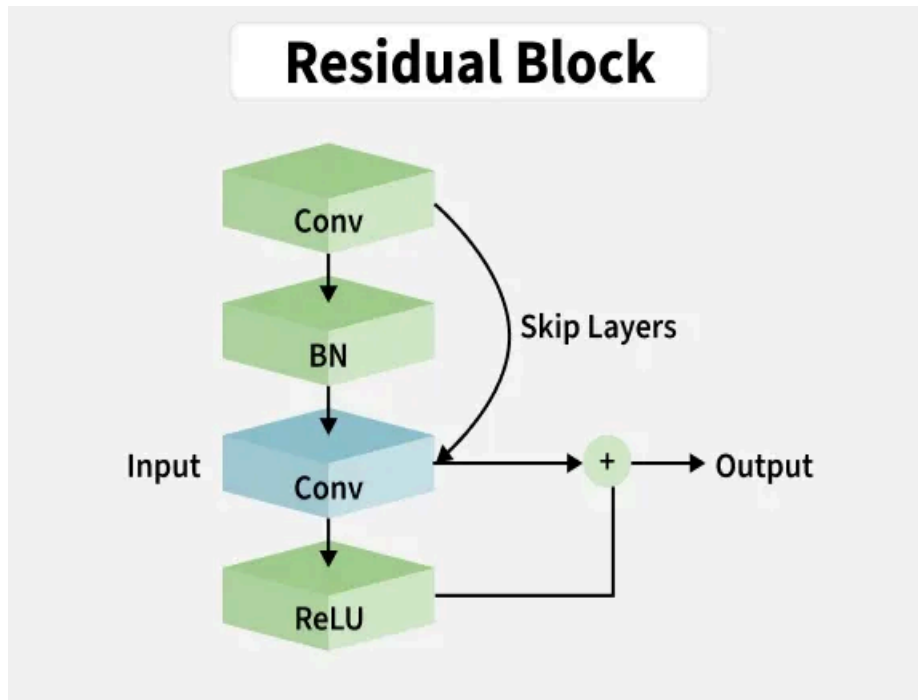
1. ResNet -

Residual Networks (ResNet) is a deep learning architecture designed to enable efficient training of very deep neural networks.

It introduces skip connections (Residual Blocks), which allow the model to learn residual mappings instead of direct transformations.

Hence it helps preventing vanishing gradient problems in very deep models, as accuracy degrades if traditional network goes into deep network,

It helps in building networks with hundreds or even thousands of layers.



2. Efficient-Net -

EfficientNet is a family of convolutional neural networks (CNNs) that aims to achieve high performance with fewer computational resources. It uses a highly optimized baseline architecture.

Compound Scaling Method -

EfficientNet internally uses a compound scaling method, which involves simultaneous adjustment of network width, depth, and resolution using a set of fixed scaling coefficients. This approach ensures that the model adapts seamlessly to varying computational constraints while preserving its performance across different scales and tasks.

A compound coefficient ϕ to uniformly scales network width, depth, and resolution in a principled way :

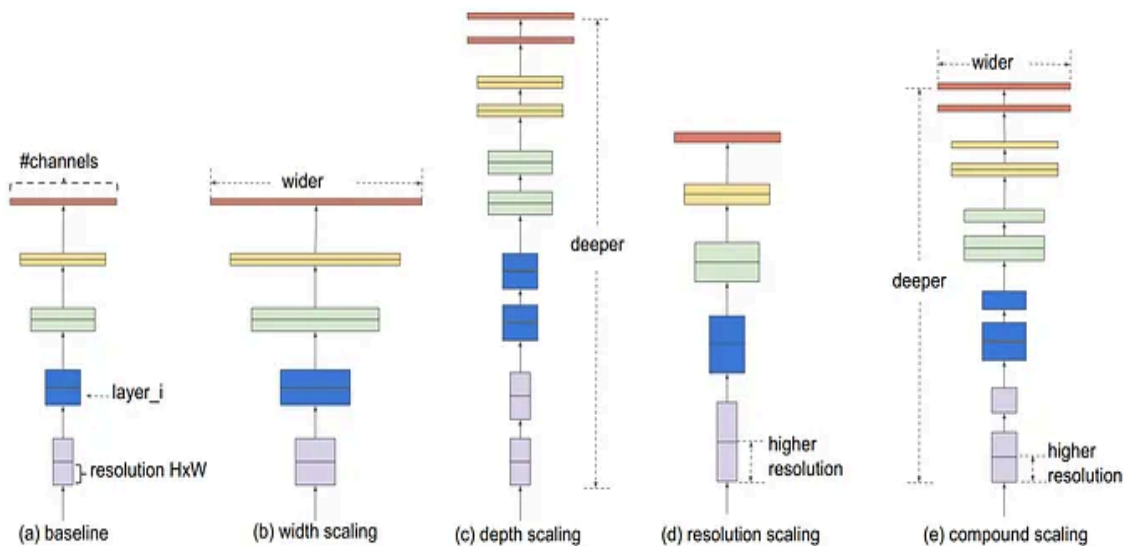
$$\text{depth: } d = \alpha^\phi$$

$$\text{width: } w = \beta^\phi$$

$$\text{resolution: } r = \gamma^\phi$$

$$\text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

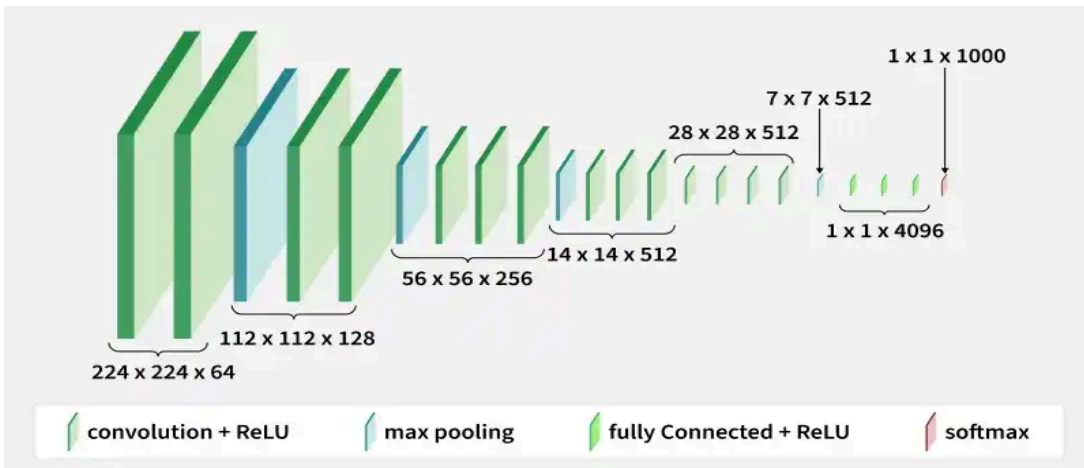


3.VGG-16 -

The VGG-16 architecture is a deep convolutional neural network (CNN) designed for image classification.

It is known for its simple and uniform structure and it consists of 16 layers (13 convolutional + 3 fully connected) arranged in blocks, where convolutional layers are followed by max-pooling for downsampling.

It is trained on input image size of an rgb (224, 224, 3) and it relies on kernels of size (3,3).



B. TCN (Temporals Convolutional Network) -

TCN usually refers to Temporal Convolutional Networks, just like 1D convolutional neural networks designed for sequential data and time-series forecasting. Since Python's standard library does not include built-in TCN layers, we have to rely on popular open-source packages like keras-tcn, pytorch-tcn.

Core Architecture of TCN -

TCNs replace Recurrent Neural Networks (RNNs) and its modified versions (LSTMs or GRUs) by offering faster, parallelized training loops. Its implementation relies on three structural principles :

- **Causal Convolutions :** Ensures the model cannot read future data points when predicting the present state.

When we are predicting state at time t , the network can only look for $(t-1)$, $(t-2)$, etc points. Hence it prevents the network from looking into the future.

- Dilated Convolutions : A dilated convolutional network skips points at an exponentially increasing rate.

Ex- Layer 1 (Dilation 1) : Looks for adjacent points (1,2,3).

Layer 2 (Dilation 2) : Now in this layer it will skip points (1,3,5).

Layer 3 (Dilation 4) : Now in this layer it will skip more points (1,5,14).

.....

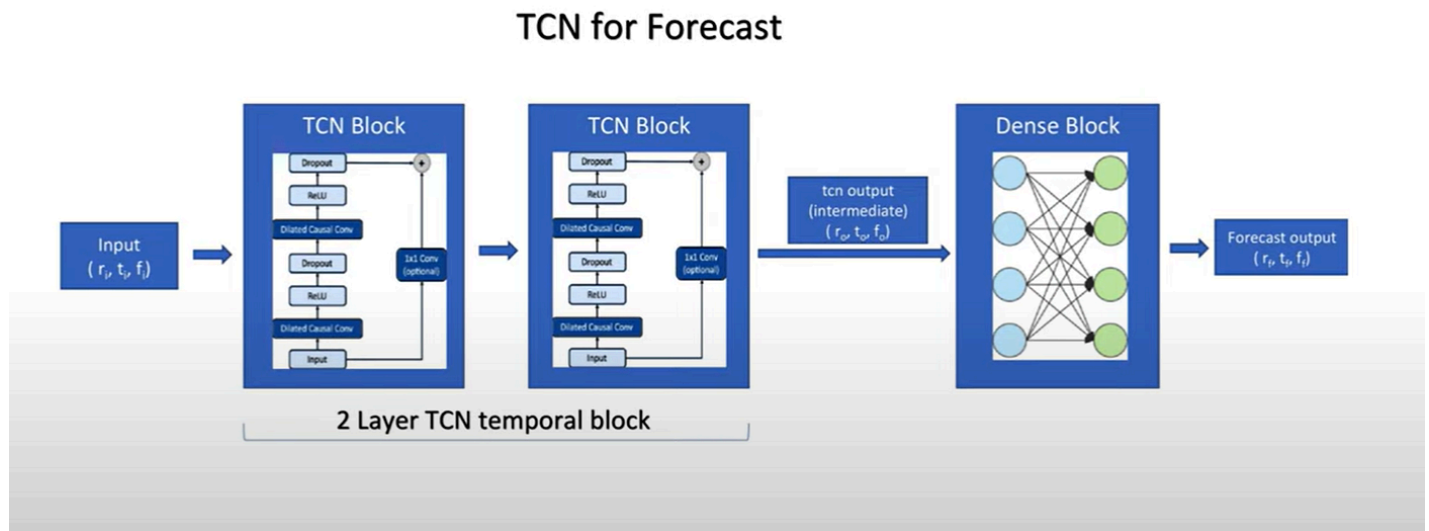
Layer 10 (Dilation 512) : Now in this layer it will skip massive chunks of points.

- Residual Blocks : Employs skip-connections to allow deep layers without encountering vanishing gradient bugs.

Why TCNs not RNNs ?

As we know RNNs are very useful in time series or sequential data so why are we going with TCNs, the reason if suppose we want to analyze 12,000 points a second, RNNs models read data

sequently one after another so reading that point will take forever to complete and it will mathematically forgotten what happened at point 1, RNNs also suffer from vanishing gradient over a long window. TCNs were invented basically to solve this problem.



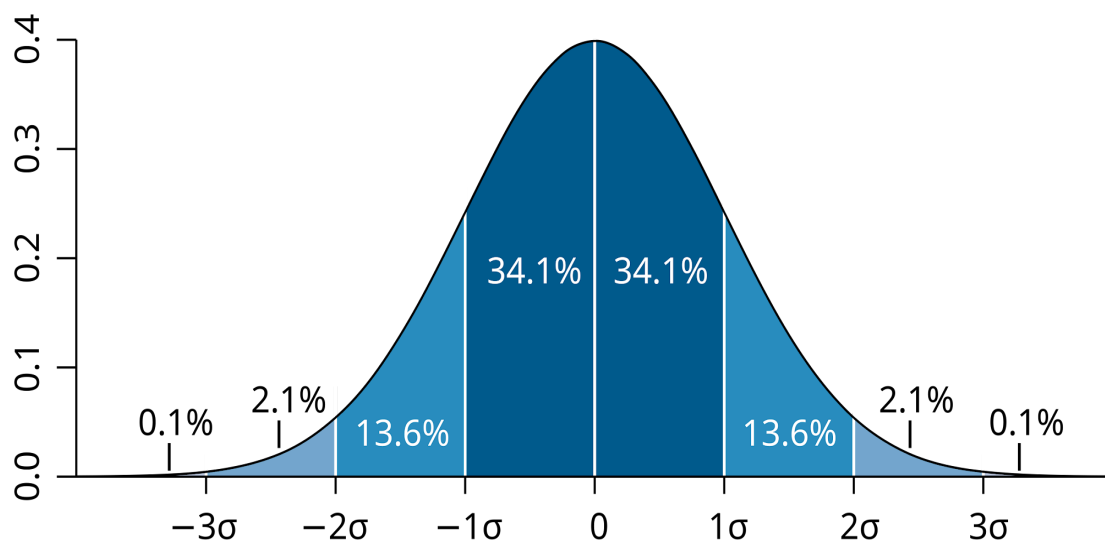
How TCNs will help us in analyzing the signals ?

TCN is just like a 1D CNN but it has additional features that will help us in analyzing 1D signals more efficiently than CNN. TCNs will be used to process the raw 1D time-series sequences directly. TCNs are just dilated causal convolutions, its architecture maintains the strict temporal ordering of the signal while exponentially expanding its receptive field, allowing the network to simultaneously capture instantaneous high-frequency fault impacts and longer-term mechanical degradation trends.

C. Normalizing flows -

This is a Unsupervised learning AI approach used for Anomaly Detection. In the real world, we have millions of hours of healthy machine data, but very little faulty data. We train a Normalizing Flow purely on the healthy data to learn its exact mathematical probability. Then, if a broken machine generates a signal, the AI flags it because it falls outside that learned "normal" probability distribution.

We will train the model on baseline signals of healthy bearings. By utilizing a sequence of invertible neural network transformations, the normalizing flow will map a simple base distribution probability density of mechanical state.



Assuming it is a learned normal distribution and if a fault data is tested against it then it will fall outside of it.

Workflow and RoadMap -

Week 1 :

- learning about bearings, their structural importance, and their fault frequencies. Studying some fundamentals of AI, machine learning and structuring CWRU dataset using scipy library.

Week 2 - 3 :

- Implementing time domain statistical extraction like RMS and Kurtosis from the data set.
- Understanding the difference between time domain and frequency domain.
- Implementing the frequency domain pipeline utilizing fast fourier transform (FFT) to target BPFI and BPFO, training the models and classifying their accuracy and precision.

Week 4 :

- Studying deep learning fundamentals.
- Learning about gramian angular fields (GAFs) convert 1D signal to 2D image, implementing it for one window to strong fundamentals and how it works.
- Learning about Pytorch, framework, Tensors, Dataset and Dataloader classes, its fundamentals and how to train models on it.

Week 5 - 6 :

- Learning about Convolutional Neural Network (CNNs) and different pre-built models like Resnet, vgg-16, etc.

- How to ingest GAF tensors into CNNs neural network.
- Train the network to identify physical fault textures.

Week 7 - 8 :

- Understanding the difference between 1D CNNs and TCNs networks.
- Studying TCNs architecture.
- Studying pytorch-tcn module to implement it.
- Feeding 1D raw signals to TCNs directly and training the model.

Week 9 :

- Understanding the need of unsupervised learning.
- How normalization will help us in anomaly detection.
- Train Normalizing flows on normal healthy data to establish probability distribution and testing it against fault signals.
- Optimize model, fix errors and compare accuracy, computational efficiency of various models.

REFERENCES :

- [https://en.wikipedia.org/wiki/Bearing_\(mechanical\)](https://en.wikipedia.org/wiki/Bearing_(mechanical))
- <https://informaticsjournals.co.in/index.php/jmmf/article/view/32937/22016>
- <https://medium.com/@RashmiW/vibration-analysis-of-bearings-for-early-fault-detection-a-matlab-case-study-1e2ff244c78e>
- <https://www.mdpi.com/2410-390X/9/4/33>
- https://scholarcommons.scu.edu/cgi/viewcontent.cgi?article=1027&context=eng_grad
- <https://www.sciencedirect.com/science/article/pii/S2949822826004570>
- <https://www.geeksforgeeks.org/deep-learning/deep-learning-tutorial/>
- <https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/>
- <https://medium.com/@rehan020345/understanding-temporal-convolutional-networks-tcns-from-cnn-basics-to-full-sequence-mastery-57d0804ad8c8>
- <https://medium.com/@sahaj432/converting-non-stationary-signals-into-2d-images-for-deep-learning-applications-de79d302c54a>
- https://pyts.readthedocs.io/en/stable/auto_examples/image/plot_single_gaf.html
- <https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/>
- <https://arxiv.org/abs/1512.03385>
- <https://arxiv.org/abs/1409.1556>
- <https://arxiv.org/abs/1905.11946>

- <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10145440>
- <https://arxiv.org/pdf/2602.09699>
- <https://www.mdpi.com/2075-1702/10/6/459>
- <https://www.mdpi.com/2076-3417/10/5/1802>
- <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=11045386>
- <https://ieeexplore.ieee.org/document/4304127>

Task :

Code -

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.io
from scipy.stats import kurtosis
from scipy.fft import fft, fftfreq

normal_data = scipy.io.loadmat('97_Normal_0.mat')
fault_data = scipy.io.loadmat('OR007_6_1_136.mat')

print("Keys in normal Data", normal_data.keys())
print("Keys in fault Data", fault_data.keys())

normal_signal = normal_data['X097_DE_time'].flatten()[:4096]
fault_signal = fault_data['X136_DE_time'].flatten()[:4096]
```



```
rate = 12000
normal_freq = fftfreq(4096, 1/rate)[:2048]
fault_freq = fftfreq(4096, 1/rate)[:2048]
normal_fft = 2.0/4096 * np.abs(fft(normal_signal)[:2048])
fault_fft = 2.0/4096 * np.abs(fft(fault_signal)[:2048])

normal_mean = np.mean(normal_signal)
normal_var = np.mean((normal_signal - normal_mean)**2)

fault_mean = np.mean(fault_signal)
fault_var = np.mean((fault_signal - fault_mean)**2)

normal_rms = np.sqrt(np.mean(normal_mean**2))
fault_rms = np.sqrt(np.mean(fault_mean**2))

normal_kurt = np.mean((normal_signal -
normal_mean)**4)/(normal_var**2)
fault_kurt = np.mean((fault_signal - fault_mean)**4)/(fault_var**2)

#using scipy.stats
# normal_kurt = kurtosis(normal_signal)
# fault_kurt = kurtosis(fault_signal)

normal_ptp = np.ptp(normal_signal)
fault_ptp = np.ptp(fault_signal)

normal_amp = np.abs(normal_fft[:2048]) / 4096.0
fault_amp = np.abs(fault_fft[:2048]) / 4096.0

dom_id = np.argmax(normal_amp)
```

```
normal_dom_freq = normal_freq[dom_id]
```

```
dom_id = np.argmax(fault_amp)  
fault_dom_freq = fault_freq[dom_id]
```

```
def status(rms, kurt):  
    if kurt < 3.5 and rms < 0.3:  
        return "Healthy"  
    else:  
        return "Faulty"
```

```
print("Normal Machine    Faulty Machine")  
print(f"Dominant frequency : {normal_dom_freq:.4f}    {fault_dom_freq:.4f}")  
print(f"RMS : {normal_rms:.4f}    {fault_rms:.4f}")  
print(f"kurtosis : {normal_kurt:.4f}    {fault_kurt:.4f}")  
print(f"peak to peak : {normal_ptp:.4f}    {fault_ptp:.4f}")  
print(f"Health Status : {status(normal_rms, normal_kurt)}    {status(fault_rms,  
fault_kurt)}")
```

```
plt.figure(figsize=(15,10))
```

```
plt.subplot(2,2,1)  
plt.plot(normal_freq, normal_fft, color='blue')  
plt.title("Normal Bearing Vibration")  
plt.xlabel("Data Points (freq)")  
plt.ylabel("Amplitude")  
plt.ylim(0,0.5)
```

```
plt.subplot(2,2,2)  
plt.plot(fault_freq, fault_fft, color='red')
```

```
plt.title("Fault Bearing Vibration")
plt.xlabel("Data Points (freq)")
plt.ylabel("Amplitude")
plt.ylim(0,0.5)
```

```
plt.subplot(2,2,3)
plt.plot(normal_signal, color='blue')
plt.title("Normal Bearing Vibration")
plt.xlabel("Data Points (Time)")
plt.ylabel("Amplitude")
plt.ylim(-2,2)
```

```
plt.subplot(2,2,4)
plt.plot(fault_signal, color='red')
plt.title("Fault Bearing Vibration")
plt.xlabel("Data Points (Time)")
plt.ylabel("Amplitude")
plt.ylim(-2,2)
```

```
plt.tight_layout()
plt.show()
```

Output -

```
Keys in normal Data dict_keys(['__header__', '__version__', '__globals__', 'X097_DE_time', 'X097_FE_time', 'X097RPM'])
Keys in fault Data dict_keys(['__header__', '__version__', '__globals__', 'X136_DE_time', 'X136_FE_time', 'X136RPM'])
Normal Machine Faulty Machine
Dominant frequency : 1037.1094 717.7734
RMS : 0.0124 0.0083
kurtosis : 2.8915 6.8297
peak to peak : 0.4959 9.9572
Health Status : Healthy Faulty
```

Plots -

